



10. Deploying Neural Network Models

"You don't start from a dataset, you start from a problem."

Previous Section:

 [9. Evaluating & Improving Neural Network Models](#)

Contents:

[1. Defining a Task for Deep Learning](#)

- [1.1. Frame the Problem](#)
- [1.2. Existing Solutions](#)
- [1.3. Data Annotation](#)
- [1.4. Representative Data](#)
- [1.5. Concept Drift](#)
- [1.6. Sampling Bias](#)
- [1.7. Evaluation Metrics](#)

[2. Developing a Model](#)

- [2.1. Preparing the Data](#)
- [2.2. Choosing an Evaluation Protocol](#)
- [2.3. Beating a Baseline](#)
- [2.4. Scaling the Model Until It Overfits](#)
- [2.5. Regularizing and Tuning](#)

[3. Deploy the Model](#)

- [3.1. What Deployment Actually Means](#)
- [3.2. Communicating Results to Stakeholders](#)

[3.3. Choosing a Production Environment](#)

[3.4. Model Optimization for Inference](#)

[3.5. Thresholds and Trade-offs](#)

[3.6. Monitoring in Production](#)

[3.7. A/B Testing and Validation](#)

[3.8. Model Maintenance and Retraining](#)

[3.9. Lifecycle View of Deployment](#)

[4. Training an Inference Model](#)

[4.1. Data Generation \(Simulated Reality\)](#)

[4.2. Data Preparation \(Defining Meaning\)](#)

[4.3. Input Construction \(Model Interface Layer\)](#)

[4.4. Class Weighting \(Handling Imbalance\)](#)

[4.5. Model Construction \(Neural System Design\)](#)

[4.6. Training Phase \(Learning from Experience\)](#)

[4.7. Inference \(Deployment Simulation\)](#)

[4.8. Evaluation \(Measuring Understanding\)](#)

[4.9. Deployment View \(Unseen Data\)](#)

[4.10. Training Dynamics Visualization \(Learning Behavior Analysis\)](#)

[Summary](#)

[References](#)

When most people begin learning Deep Learning, their attention immediately goes to the models themselves. They focus on architectures, layers, optimization, and the transition from traditional Machine Learning into Neural Networks. But there is one important reality that many beginners overlook:

A powerful model means nothing if you do not understand where and how it can actually be used.

Building a Neural Network is only one part of the journey. In real-world applications, Deep Learning is not just about training models inside notebooks or achieving high accuracy scores. The true objective is to create systems that solve real problems for real users.

This is why the workflow of Deep Learning is usually viewed as a complete pipeline rather than a single task. Broadly speaking, the process can be divided into three major stages:

1. **Define the task:** Before building anything, we first need to understand the problem itself. What exactly are we trying to solve? What does the customer or business actually need?
This stage involves understanding the domain, collecting data, analyzing what the data represents, and deciding how success will be measured.
2. **Develop the model:** Once the problem is clearly defined, we begin constructing the learning system. This includes preparing the data, selecting evaluation methods, creating baseline models, training Neural Networks, and improving their generalization performance through tuning and regularization. This is the stage most learners are familiar with because it contains the mathematics, optimization, and architecture design commonly associated with Deep Learning.
3. **Deploy the model:** And finally, we arrive at the stage that many people underestimate: deployment. A model is only useful when it can operate in the real world. After development, the model may be deployed to web servers, mobile applications, embedded systems, or cloud platforms where users can interact with it directly.

But deployment is not simply about uploading a model and walking away. Once a model is released into the real environment, new challenges begin to appear:

- Real-world data may look different from training data
- User behavior may change over time
- The model's performance can gradually decline
- New data must continuously be collected for future improvements

This transforms Deep Learning into a continuous cycle rather than a one-time project. We define problems, build models, deploy them, observe how they behave in the real world, and then improve them again.

So in this section, we are not just studying Neural Networks as mathematical structures. We are studying how Neural Networks become actual products and intelligent systems that people use every day.

1. Defining a Task for Deep Learning

One of the biggest misconceptions beginners have about Deep Learning is this: They think everything starts from the data. But in reality, it starts from the problem. Always.

Before collecting datasets, training Neural Networks, or tuning hyper-parameters, we first need to understand what problem we are actually trying to solve. Because Deep Learning is not just mathematics and models. It is a tool created to solve real-world problems.

1.1. Frame the Problem

And the moment we start working with real-world problems, another important issue immediately appears: Ethics. Technology is not neutral. A Machine Learning system can affect people's decisions, opportunities, privacy, and even safety. Some problems naturally contain ethical risks. Some datasets already contain human bias. And if we are not careful, the model may amplify those biases at massive scale.

This is why building Deep Learning systems is not only a technical process. It is also a social responsibility. Before building any model, we must deeply understand the context surrounding the problem. Some important questions must always be asked first:

- What problem are we trying to solve?
- Who will use the system?
- What value does the system create?
- What data currently exists?
- Can new data realistically be collected?

The goal here is simple: **Transform a real-world problem into a clearly defined Machine Learning problem.**

And to do that, we need to define several critical components:

- What will be the input data?
- What exactly are we trying to predict?
- Is Machine Learning even the correct solution?

That last question is more important than many beginners realize. Not every problem needs Machine Learning (let alone Deep Learning).

1.2. Existing Solutions

Sometimes a simple rule-based system is faster, cheaper, and easier to maintain. Sometimes humans already solve the task effectively without AI. In many real projects, the objective of ML is not to replace the current workflow entirely, but to improve or support it.

So before building a model, we should investigate the existing process:

- How is the problem currently solved?
- Is there already a rule-based system?
- Are humans manually handling the task?
- What weaknesses exist in the current solution?

Only after understanding these limitations can we determine whether Deep Learning is truly useful.

Next comes another major step: identifying the type of Machine Learning problem itself. Different problems require different approaches. For example:

- Binary Classification → spam or not spam
- Multiclass Classification → identifying one class among many
- Regression → predicting continuous values like house prices
- Ranking → sorting search results or recommendations
- Recommendation Systems → suggesting products or videos
- Image Segmentation → classifying every pixel in an image

At this stage, we also begin asking practical questions:

- Are we predicting one label or multiple labels?
- Is the data clean or noisy?
- Is the dataset balanced?
- Is enough information available to learn meaningful patterns?

And this leads to an important realization. Whenever we formulate a Machine Learning problem, we are secretly making two assumptions:

1. The target can actually be predicted from the input data.
2. The dataset contains enough information to learn the relationship.

These assumptions sound obvious, but they are not always true. Some problems simply cannot be solved effectively using Machine Learning. Sometimes the signal does not exist. Sometimes the data is incomplete. Sometimes the target itself is inconsistent. This is why experimentation and validation are necessary.

1.3. Data Annotation

Now let's talk about something many researchers quickly discover: Collecting data is often the hardest part of the entire project. Not training. Not modeling. Data collection.

Building a high-quality dataset can consume enormous amounts of time, money, and human effort. The dataset must match both the input and the target correctly. And in many cases, we may need to collect entirely new data; manually label samples; clean corrupted data; remove duplicates; verify annotation quality

A good dataset is one of the most valuable assets in any Machine Learning project. For supervised learning especially, annotation becomes extremely important because the model learns directly from labeled examples.

There are many ways to obtain annotations:

- Label the data yourself
- Use crowdsourcing platforms
- Hire professional annotation companies
- Work with domain experts

And here is the critical truth: The quality of the annotation often determines the quality of the model itself. Even with a powerful Neural Network, poor labels usually produce poor predictions.

1.4. Representative Data

Another principle that beginners often underestimate is this: ***Training data must resemble production data.***

A model trained on clean laboratory data may completely fail in the real world if the production environment looks different. This is why we should collect data from real deployment environments whenever possible. Avoid datasets that are unrealistically perfect but disconnected from reality. Because in Machine Learning, realism matters more than cleanliness.

1.5. Concept Drift

And even if the model works today, another problem eventually appears: concept drift. Concept drift occurs when production data changes over time. As user behavior changes, environments evolve, or trends shift, the original data distribution slowly becomes outdated. When that happens, model performance gradually decreases.

This is extremely common in systems such as: Recommendation systems; Fraud detection; User-generated content platforms; Text classification systems. Because of this, Deep Learning systems are rarely static. Data must continuously be collected, monitored, and used for retraining.

1.6. Sampling Bias

Another dangerous issue is sampling bias. Sampling bias happens when the collected dataset does not properly represent the real population.

This can occur because: Data collection methods are biased; Certain user groups are missing; Some environments are underrepresented

And when the dataset itself is biased, the model learns incorrect conclusions. This is why we should never treat datasets as black boxes. Before training any model, we must explore the dataset carefully:

Inspect image or text samples → Visualize histograms → Check missing values → Detect class imbalance → Search for target leakage

The goal is to understand the weaknesses of the data before modeling even begins.

1.7. Evaluation Metrics

Finally, we arrive at one of the most important principles in Machine Learning: If you want to optimize something, you must be able to measure it.

In many beginner projects, we use validation accuracy and loss because they are easy to understand. But in real-world systems, metrics must match the actual business objective.

Different tasks require different evaluation metrics: Accuracy; Precision; Recall; ROC AUC; Weighted Accuracy; Custom Metrics

And ideally, technical metrics should connect directly to real-world value. Because at the end of the day, a model with excellent mathematical performance but no practical impact is not truly solving the problem.

2. Developing a Model

Once we understand the problem and the dataset, we can finally begin developing the model itself. This is usually the stage people imagine when they hear the words "Deep Learning." Neural Networks, layers, optimization, backpropagation, GPUs, training loops. But in reality, developing a successful model is not just about choosing a fancy architecture.

It is a structured process. Broadly speaking, this stage consists of several major steps:

- Preparing the data
- Choosing an evaluation protocol
- Beating a baseline
- Scaling the model until it overfits
- Regularizing and tuning for generalization

Let's walk through each of them carefully.

2.1. Preparing the Data

One important thing beginners quickly discover is this: Deep Learning models usually cannot process raw data directly. Neural Networks expect numerical representations, usually in the form of vectors or tensors. Because of that, raw data must first be transformed into a format the model can understand.

This preparation stage often includes: Vectorization; Normalization; Handling missing values; Feature engineering if necessary

The objective is simple: Transform messy real-world data into structured numerical tensors suitable for Neural Networks.

Vectorization

The first major step is vectorization. Vectorization means converting data into vectors or tensors that can be processed mathematically by the model.

Most Neural Networks operate on real-valued tensors, sometimes integer tensors, occasionally sequence-based representations

Different types of data require different vectorization strategies:

- Text → integer sequences or one-hot vectors
- Images → pixel tensors
- Tabular data → feature matrices

For example, a color image is not “seen” by the Neural Network as an actual image. Instead, it becomes a tensor of pixel values.

Similarly, text is not interpreted as language directly. It becomes sequences of numbers representing words, tokens, or embeddings.

This is the very first step that allows the model to “read” the data.

Normalization

After vectorization, another important step appears: normalization.

Normalization transforms input data into a scale that allows the Neural Network to learn more stably and efficiently.

Without normalization, features with extremely large values may dominate smaller features, making optimization unstable.

In practice, good training data usually has two important properties: Values remain relatively small; Features exist within similar ranges

Common normalization strategies include:

- Scaling values into the range $[0, 1]$ → **Min-Max Scaling**
- Standardization with: Mean = 0; Standard deviation = 1 → **Z-Score**

This helps gradient descent converge faster and improves training stability.

Handling Missing Values

Real-world data is rarely perfect. Missing values appear constantly in production systems, databases, surveys, logs, and user-generated content.

Handling missing values correctly is extremely important because Neural Networks do not naturally “understand” missing information.

- For categorical features: We may create a dedicated “missing” category
- For numerical features: Randomly filling zeros is usually a bad idea. We may use mean imputation, median imputation, or another predictive model to estimate missing values

And here is an important principle: If missing values exist in production data, the model should experience missing values during training as well. Otherwise, deployment behavior may become unstable.

2.2. Choosing an Evaluation Protocol

After preparing the data, we need a reliable way to evaluate the model. The goal of evaluation is not merely measuring training performance. The true goal is estimating how well the model will perform on unseen production data.

Several evaluation protocols are commonly used: Holdout validation; K-fold cross-validation; Iterated K-fold validation. The correct choice depends on Dataset size; Computational cost; Required reliability

Regardless of the protocol, one principle always remains important: The validation data must represent production data as closely as possible. Otherwise, evaluation results become misleading.

2.3. Beating a Baseline

One of the most overlooked stages in Deep Learning is establishing a baseline. Beginners often rush directly into complex architectures without asking an important question:

■ “Is the model actually learning anything meaningful?”

The first objective is not perfection. The first objective is statistical power.

A small Neural Network should at least outperform a simple baseline solution. This stage often focuses on feature engineering, architecture priors, and training configuration

If the model cannot beat a simple baseline, something may be fundamentally wrong:

- The dataset may be poor
- The problem formulation may be incorrect
- The target may not be predictable
- The features may contain insufficient information

At this stage, loss functions also become important. A proper loss function should be computable on mini-batches and be differentiable for backpropagation

Different tasks use different output activations and losses:

- Sigmoid → binary classification
- Softmax → multiclass classification
- Sometimes combinations of both → multi-label classification

The choice of loss directly affects how the model learns.

2.4. Scaling the Model Until It Overfits

Once the model successfully beats the baseline, we perform an interesting experiment: We intentionally try to make it overfit. This may sound strange at first.

But overfitting proves that the model has enough capacity to memorize the training data. If the model cannot even overfit, it may simply be too weak.

Common ways to increase capacity include:

- Adding more layers
- Increasing the number of units
- Increasing parameter count
- Training for more epochs

During this stage, we monitor Training loss and Validation loss. Eventually, a pattern appears:

- Training performance keeps improving
- Validation performance begins decreasing

This is the moment the model starts overfitting. And that transition tells us the model now has sufficient learning capacity.

2.5. Regularizing and Tuning

Once overfitting appears, the next goal becomes maximizing generalization. Generalization is the true objective of Deep Learning. We do not want a model that only memorizes training examples. We want a model that performs well on unseen data.

To improve generalization, we experiment with different strategies:

- Modifying the architecture
- Adding or removing layers
- Using dropout
- Applying L1 or L2 regularization
- Tuning learning rates
- Adjusting batch sizes
- Changing the number of units
- Improving features or dataset quality

This process becomes iterative: ***Train*** → ***Validate*** → ***Adjust*** → ***Repeat***. And this cycle may continue many times during real-world projects.

However, there is another danger hidden here: validation leakage. If we repeatedly tune the model using the same validation set, the model indirectly begins "learning" from validation feedback.

Even though the model never trains directly on validation samples, repeated adjustments slowly bias the system toward that validation set. When this happens, validation scores stop being trustworthy. This is why, after selecting the final architecture, we usually **retrain the model on both training and validation data and evaluate only once on the test set**

The test set should remain untouched until the very end. And if test performance is significantly worse than validation performance, it often indicates problems with the evaluation protocol itself.

In the end, the objective of model development is not simply achieving low loss or high accuracy. The true goal is much more important: Achieving strong generalization on unseen data.

3. Deploy the Model

When a model successfully passes evaluation on the test set, the next phase is deployment. This is the stage where the model moves from a controlled environment into real-world usage.

Deployment is not simply saving a trained model file. It is the process of making the model usable, reliable, and maintainable in production systems.

3.1. What Deployment Actually Means

In production, a model becomes a component of a larger system. It must:

- Accept real-world input data
- Produce predictions under constraints (time, memory, scale)
- Integrate with applications or services
- Operate reliably over time

At this stage, the model is no longer an experiment. It becomes part of a product.

3.2. Communicating Results to Stakeholders

Before launch, model performance must be interpreted in business terms, not only technical metrics.

Instead of reporting: **"Accuracy = 98%"**. We translate results into meaningful impact:

- False positives: How often does the model make incorrect positive predictions?
- False negatives: How often does it miss important cases?
- Manual review load: How many cases still require human checking?
- Business impact: Cost, risk, or benefit of model decisions

This step ensures that stakeholders understand trade-offs clearly.

3.3. Choosing a Production Environment

A production model must be adapted to its deployment environment. Common options include: Cloud services (APIs, scalable servers); Edge devices (smartphones, IoT, embedded systems); Browser-based inference (JavaScript models); Desktop applications

Each environment imposes constraints: Latency requirements (how fast predictions must be); Memory usage; Compute limitations; Connectivity availability

3.4. Model Optimization for Inference

Training models are not always suitable for deployment. In production, we often optimize them by:

- Reducing model size
- Quantization (lower precision computation)

- Pruning unnecessary parameters
- Converting to efficient formats (ONNX, TensorFlow Lite, TorchScript)

The goal is to improve speed and efficiency without significantly reducing performance.

3.5. Thresholds and Trade-offs

Most models do not output a final decision directly. Instead, they output probabilities or scores. A decision threshold determines how predictions are interpreted.

Changing this threshold affects: Precision vs recall; False positives vs false negatives; Business cost balance

Choosing the correct threshold is a critical deployment decision, not just a modeling choice.

3.6. Monitoring in Production

Once deployed, model behavior must be continuously tracked.

Key aspects to monitor:

- Prediction quality over time
- Data distribution changes (data drift)
- System performance (latency, failures)
- Business metrics linked to predictions
- Edge cases and failure patterns

Monitoring ensures the model remains reliable in changing environments.

3.7. A/B Testing and Validation

Before full rollout, models are often tested in controlled environments:

- A/B testing compares a new model with an existing one

- Traffic is split between versions
- Performance is evaluated in real conditions

This reduces risk before full deployment.

3.8. Model Maintenance and Retraining

Real-world data is not static. Over time, user behavior changes, new patterns appear, old patterns become outdated, This leads to model degradation, often called data drift or concept drift.

To handle this, models must be periodically retrained with new data; updated or replaced when performance drops; continuously evaluated against fresh benchmarks.

3.9. Lifecycle View of Deployment

Deployment is not the end of the machine learning process. It is part of a continuous cycle:

1. Deploy model
2. Monitor performance
3. Collect new data
4. Detect drift or degradation
5. Retrain or update model
6. Redeploy improved version

This cycle ensures the model stays relevant over time.

Model deployment transforms a trained system into a real-world product component. It involves much more than saving a model:

- Translating results into business understanding
- Adapting to production constraints
- Optimizing for performance and efficiency

- Monitoring behavior continuously
- Updating the model as the world changes

A deployed model is not a finished model. It is a living system that must evolve with data, users, and environment.

4. Training an Inference Model

Now, we build a full deep learning system that connects all three stages of Machine Learning:

- **Define Task** → **Data Preparation**
- **Develop Model** → **Training & Validation**
- **Deploy Model** → **Inference on unseen data**

But instead of treating them as separate steps, we view them as a continuous system. A model is not just trained. It is *constructed, tested, and then released into reality*.

4.1. Data Generation (Simulated Reality)

Before learning begins, we first construct a controlled environment. We simulate a customer churn dataset where each customer represents a “real-world observation.”

The goal is simple: Learn whether a customer will churn in the next 30 days.

We use a probabilistic function (logit → sigmoid) to define the hidden truth behind the data.

```
import numpy as np
import pandas as pd

# Sigmoid: transforms raw score into probability
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Generate synthetic customer dataset
```

```

def generate_customer_data(n_samples=6000, production_shift
=False, random_state=42):

    rng = np.random.default_rng(random_state)

    plan_type = rng.choice(["free", "basic", "pro", "enterp
rise"],
                           size=n_samples, p=[0.25, 0.40,
0.25, 0.10],)

    plan_risk = pd.Series(plan_type).map({"free": 0.55, "ba
sic": 0.25,
                                         "pro": -0.05, "en
terprise": -0.35,}).values

    days_since_signup = rng.gamma(2.5, 80, n_samples).clip
(1, 900)
    discount_used = rng.binomial(1, 0.30, n_samples)
    visits_last_30d = rng.poisson(12, n_samples).astype(flo
at)
    avg_session_minutes = rng.gamma(2.2, 8, n_samples).clip
(0.5, 120)
    feature_adoption_score = rng.beta(2.0, 3.0, n_samples)
    email_click_rate = rng.beta(1.5, 8.0, n_samples)
    support_tickets = rng.poisson(1.0, n_samples).astype(fl
oat)

    if production_shift:
        visits_last_30d = np.maximum(0, visits_last_30d
- rng.poisson(2, n_sam
ples))

        feature_adoption_score = np.clip(feature_adoption_s
core
                                         - rng.normal(0.08,
0.03, n_samples), 0, 1)

        support_tickets = support_tickets + rng.poisson(0.

```

```

4, n_samples)

    risk_logit = (-1.25 + plan_risk - 0.055 * visits_last_30d
                  - 0.025 * avg_session_minutes - 1.8 * feature_adoption_score
                  - 1.4 * email_click_rate + 0.30 * support_tickets
                  - 0.0015 * days_since_signup + 0.25 * discount_used
                  + rng.normal(0, 0.55, n_samples))

    churn_probability = sigmoid(risk_logit)
    churned_next_30d = rng.binomial(1, churn_probability)

    return pd.DataFrame({"days_since_signup": days_since_signup,
                        "discount_used": discount_used,
                        "visits_last_30d": visits_last_30d,
                        "avg_session_minutes": avg_session_minutes,
                        "feature_adoption_score": feature_adoption_score,
                        "email_click_rate": email_click_rate,
                        "support_tickets": support_tickets,
                        "plan_type": plan_type,
                        "churned_next_30d": churned_next_30d,})

df = generate_customer_data(6000, production_shift=False)
df.head()

```

4.2. Data Preparation (Defining Meaning)

We now convert raw observations into structured learning signals.

We split: **Features** → **X**; **Target** → **y**

```
from sklearn.model_selection import train_test_split

target_col = "churned_next_30d"

X = df.drop(columns=[target_col])
y = df[target_col]

X_train_full, X_test, y_train_full, y_test = train_test_split(X, y, test_size=0.2,

stratify=y, random_state=42,)

X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full,

                                                    test_size
                                                    =0.25, stratify=y_train_full, random_state=42,)

categorical_features = ["plan_type"]
numeric_features = [c for c in X.columns if c not in categorical_features]
```

4.3. Input Construction (Model Interface Layer)

We define how raw data becomes model input.

```
def make_model_inputs(X_data):
    return {"numeric_features": X_data[numeric_features].to_numpy().astype("float32"),
            "plan_type": X_data["plan_type"].astype(str).to_numpy(),}
```

4.4. Class Weighting (Handling Imbalance)

```
import numpy as np

def compute_class_weight(y_values):
    y_array = np.asarray(y_values).astype(int)
    counts = np.bincount(y_array)
    total = counts.sum()

    return {0: total / (2 * counts[0]), 1: total / (2 * counts[1])}

class_weight = compute_class_weight(y_train)
```

4.5. Model Construction (Neural System Design)

We define a dual-input neural network:

- Numeric branch → normalized features
- Categorical branch → embedding via lookup

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

def build_keras_churn_model(adapt_df, hidden_units=(64, 32), dropout_rate=0.0):

    numeric_input = keras.Input(shape=(len(numeric_features)), name="numeric_features")

    normalizer = layers.Normalization()
    normalizer.adapt(adapt_df[numeric_features].to_numpy().
```

```

astype("float32"))
    x_num = normalizer(numeric_input)

    plan_input = keras.Input(shape=(), dtype=tf.string, name="plan_type")

    lookup = layers.StringLookup(output_mode="one_hot")
    lookup.adapt(adapt_df["plan_type"].astype(str))
    x_plan = lookup(plan_input)

    x = layers.Concatenate()([x_num, x_plan])

    for units in hidden_units:
        x = layers.Dense(units, activation="relu")(x)
        if dropout_rate > 0:
            x = layers.Dropout(dropout_rate)(x)

    output = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs={"numeric_features": numeric_input, "plan_type": plan_input},
                        outputs=output)

    model.compile(optimizer=keras.optimizers.Adam(0.001),
                  loss="binary_crossentropy",
                  metrics=[keras.metrics.BinaryAccuracy(),
                          keras.metrics.AUC(name="roc_auc",
c"),
                          keras.metrics.AUC(curve="PR", name="pr_auc")],)

    return model

```

4.6. Training Phase (Learning from Experience)

```

model = build_keras_churn_model(X_train)

early_stopping = keras.callbacks.EarlyStopping(monitor="val_
_roc_auc",
                                                mode="max",
patience=8,
                                                restore_best_
_weights=True,)

history = model.fit(make_model_inputs(X_train),
                    y_train.astype("float32"),
                    validation_data=(make_model_inputs(X_val), y_val.astype("float32")),
                    epochs=60,
                    batch_size=128,
                    class_weight=class_weight,
                    callbacks=[early_stopping], verbose=1,)

```

Overfitting is typically detected by observing the behavior of validation loss and validation accuracy during training.

- If the model continues to improve on the training set but the validation loss stops decreasing—or starts increasing—this is a strong signal of overfitting.
- Similarly, validation accuracy may initially improve, then reach a plateau and eventually decline while training accuracy keeps rising.

This divergence indicates that the model is beginning to memorize training patterns instead of learning generalizable relationships, and the optimal model is usually found at the point where validation performance is at its peak before degradation begins.

4.7. Inference (Deployment Simulation)

Now the model transitions into reality. It no longer learns. It only predicts.

```
def predict_proba(model, X_data):
    inputs = make_model_inputs(X_data)
    return model.predict(inputs, verbose=0).reshape(-1)
```

4.8. Evaluation (Measuring Understanding)

```
from sklearn.metrics import roc_auc_score, precision_score,
recall_score, f1_score

def evaluate(y_true, y_proba, threshold=0.5):
    y_true = np.asarray(y_true)
    y_pred = (y_proba >= threshold).astype(int)

    return {"roc_auc": roc_auc_score(y_true, y_proba),
            "precision": precision_score(y_true, y_pred),
            "recall": recall_score(y_true, y_pred),
            "f1": f1_score(y_true, y_pred),
            "flag_rate": y_pred.mean()}
```

4.9. Deployment View (Unseen Data)

We simulate production drift:

```
df_prod = generate_customer_data(2000, production_shift=True)

X_prod = df_prod.drop(columns=["churned_next_30d"])
y_prod = df_prod["churned_next_30d"]

proba_prod = predict_proba(model, X_prod)

metrics_prod = evaluate(y_prod, proba_prod)
metrics_prod
```


At this point, the system is complete: We did not just build a model. We constructed a **learning system**. It **observes** → **learns** → **predicts** → **degrades** → **adapts**.

This is the real meaning of deployment: A model is not finished when trained. It is finished when it survives reality.

4.10. Training Dynamics Visualization (Learning Behavior Analysis)

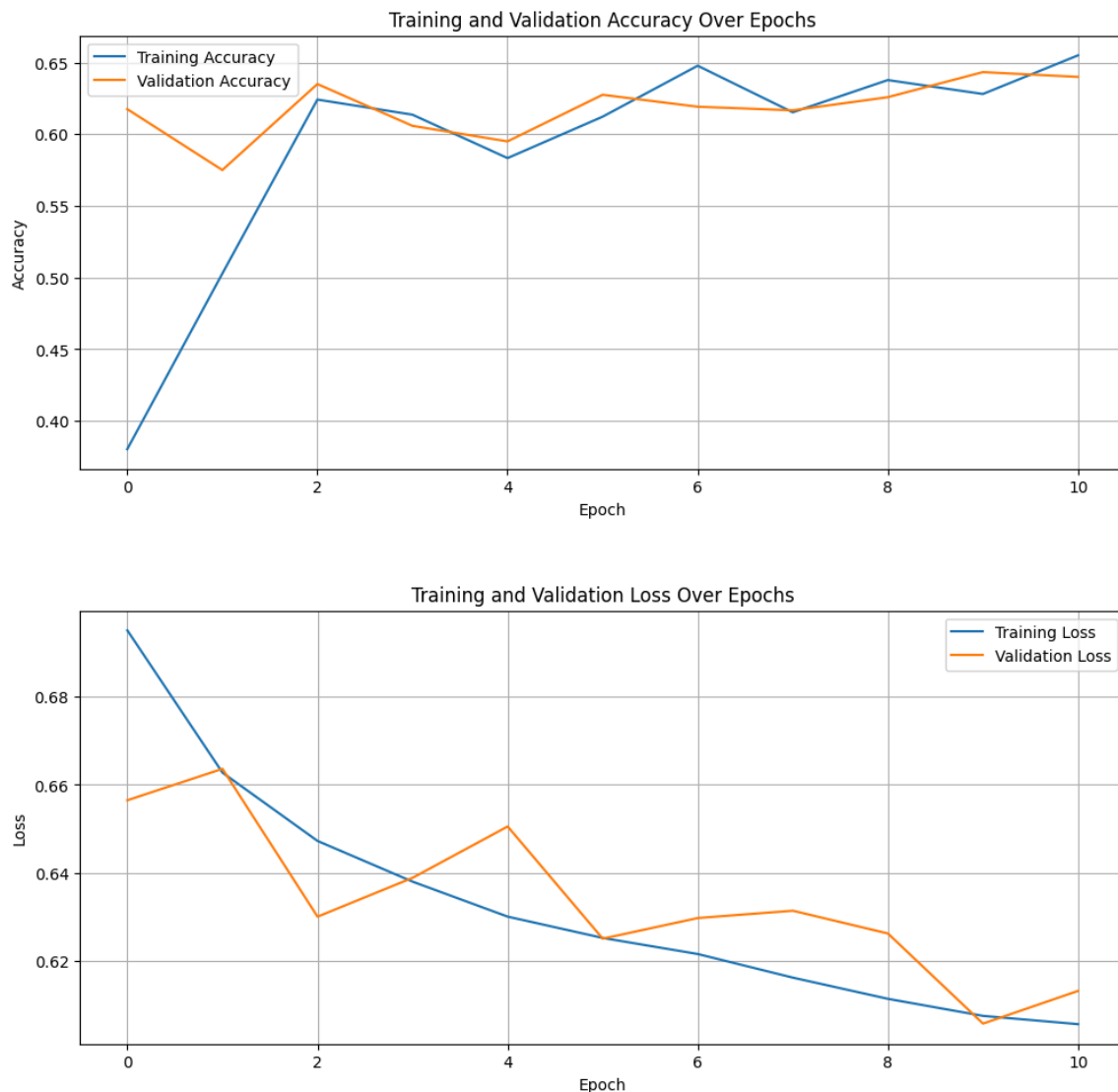
To complete the training cycle, we visualize how the model learns over time by plotting both **training and validation loss**, as well as **training and validation accuracy**.

```
import matplotlib.pyplot as plt

fig_acc = plt.figure(figsize=(12, 5))
plt.plot(history.history['binary_accuracy'], label='Training Accuracy')
plt.plot(history.history['val_binary_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

fig_loss = plt.figure(figsize=(12, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
```

```
plt.grid(True)
plt.show()
```



These curves act as a diagnostic lens into the model's behavior.

- If the training loss continues to decrease while validation loss stabilizes or increases, it signals that the model is beginning to memorize rather than generalize.
- Similarly, when validation accuracy reaches a plateau or starts to decline while training accuracy continues improving, it becomes a clear indicator that overfitting is occurring.

These visual patterns help us identify the exact point where the model transitions from learning meaningful structure to over-specializing on training

data.

Summary

This section demonstrates the complete lifecycle of deploying a neural network model—from data generation and preparation, to model construction, training, validation, and finally inference on unseen data. Rather than treating these steps as isolated components, we view them as a continuous system where each stage depends on the previous one and informs the next.

The model is not only trained to achieve high performance, but also evaluated for generalization, robustness, and stability under distribution shifts.

Through this process, we observe how a model evolves from learning patterns in controlled data to making decisions in uncertain, real-world conditions.

Ultimately, deploying a neural network is not the end of training—it is the beginning of its real existence as a functional system in production.

References

<https://medium.com/@nikhil-1e9/the-universal-workflow-of-solving-a-machine-learning-problem-d4d92cc6187f>

<https://www.youtube.com/watch?v=HevrtBl6SX4>

Next Section:

 [11. Customizing Neural Networks Models](#)